

NexChat

Product Development Report

Course:

Computer Systems and Design

Group Members:

Amathul Azeez Sarah

Hirannya Mhaisbadwe

Sanjani Kumari

Aditya Jha

August 14, 2026

Abstract

This report details the end-to-end development, architecture, and design of **NexChat**, a modern, real-time group messaging application. Built using a React frontend, a FastAPI Python backend, and native WebSockets, the application provides a seamless, low-latency chatting experience. This report discusses the product overview, UI/UX design, technology stack, and the decoupled client-server architecture developed for the Computer Systems and Design course.

Contents

1	Introduction	2
2	Overview	2
2.1	Key Product Features	2
3	Design	2
3.1	UI/UX Principles	2
3.2	Design Scenario Walkthrough	3
4	Tech Stack Used	6
4.1	Frontend (Client-Side)	6
4.2	Backend (Server-Side)	6
5	Architecture	6
5.1	WebSocket Communication Pipeline	6
5.2	System State and Resilience	6
6	Conclusion	7

1 Introduction

NexChat was conceptualized as a lightweight, highly responsive chat application allowing multiple users to connect and interact in a shared room without page reloads. The primary goal of this project was to apply core principles of computer systems engineering to create a robust, full-stack application that leverages modern web technologies—specifically native WebSockets for bidirectional communication, bypassing the limitations of traditional HTTP polling.

2 Overview

NexChat serves as a centralized group communication platform. It is designed to be highly reliable, ensuring that users do not lose their connections or chat history unintentionally.

2.1 Key Product Features

- **Real-Time Messaging:** Lightning-fast communication powered by native WebSockets ensuring zero-latency message delivery.
- **Smart Session Management:** A custom heartbeat mechanism allows users to briefly close their browser or switch tabs (within a 5-minute window) without losing their session or chat history.
- **Tab Deduplication:** Prevents ghost connections. If a user opens a new tab, the backend detects the active session token and safely disconnects the old tab, maintaining only one active connection per user.
- **Live Roster:** A real-time updating list of online users with connection status indicators.
- **Inline Replies:** Users can reply to specific messages, creating a clear conversational thread.

3 Design

The user interface was built completely from scratch, without reliance on heavy third-party CSS frameworks, to ensure maximum performance and complete design control.

3.1 UI/UX Principles

- **Design System:** We established a rigid set of CSS variables (`--clr-bg`, `--clr-accent`, etc.) to ensure visual consistency across all components.
- **Theming:** A dynamic hook intercepts theme toggles and maps a `data-theme="light"` attribute to the root HTML element, instantly flipping the application between Dark and Light modes.
- **Visual Aesthetics:** Glassmorphism was utilized in the login card via strict box-shadows, combined with CSS `mix-blend-mode` and custom keyframe animations (`@keyframes msgIn`) for smooth message appearance.

3.2 Design Scenario Walkthrough

To demonstrate the application design and user flow, we simulated a local session between two users: Aditya and Hirannya.

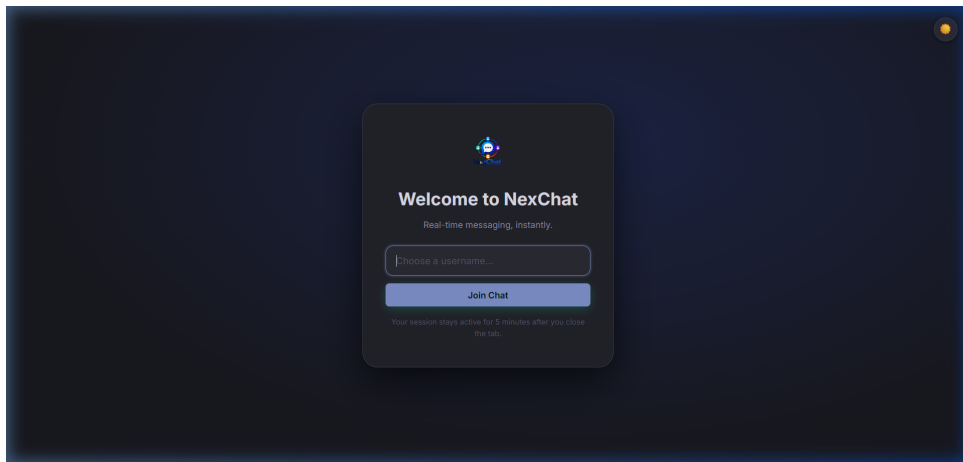


Figure 1: Step 1: The login page featuring the custom logo and glassmorphism design.

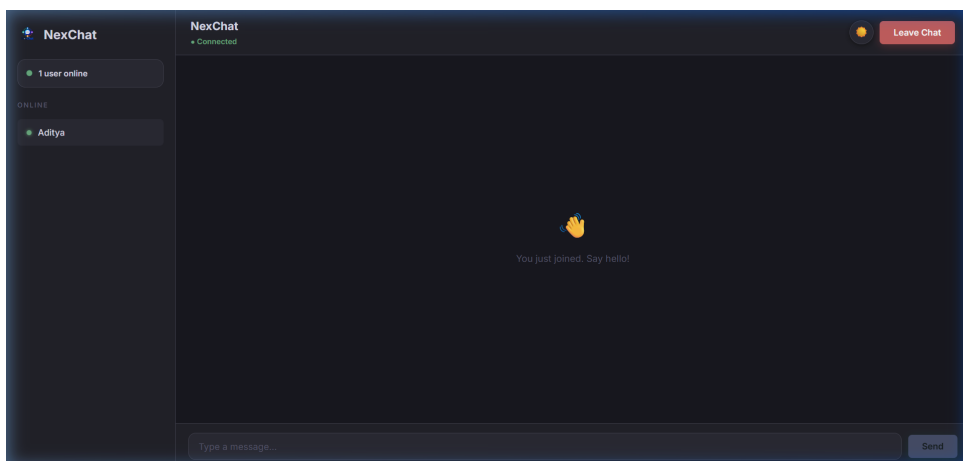


Figure 2: Step 2a: The main chat room rendered in the default Dark Mode.

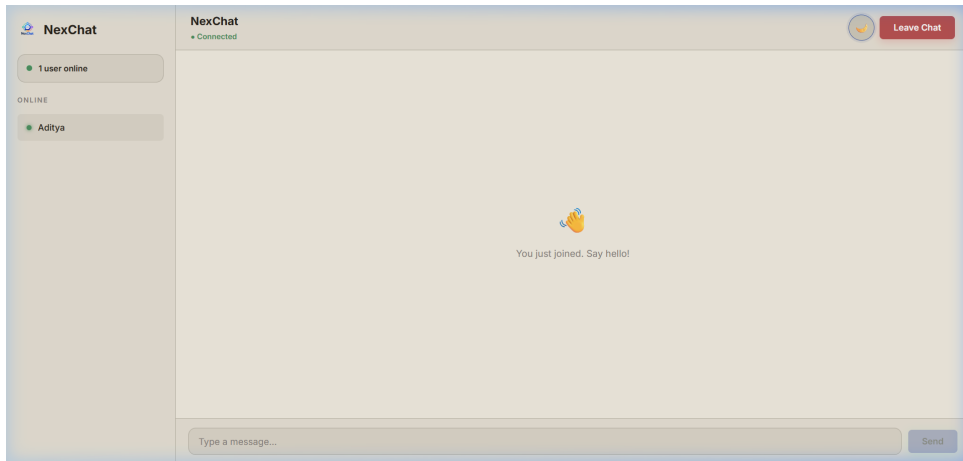


Figure 3: Step 2b: The main chat room seamlessly toggled to Light Mode.

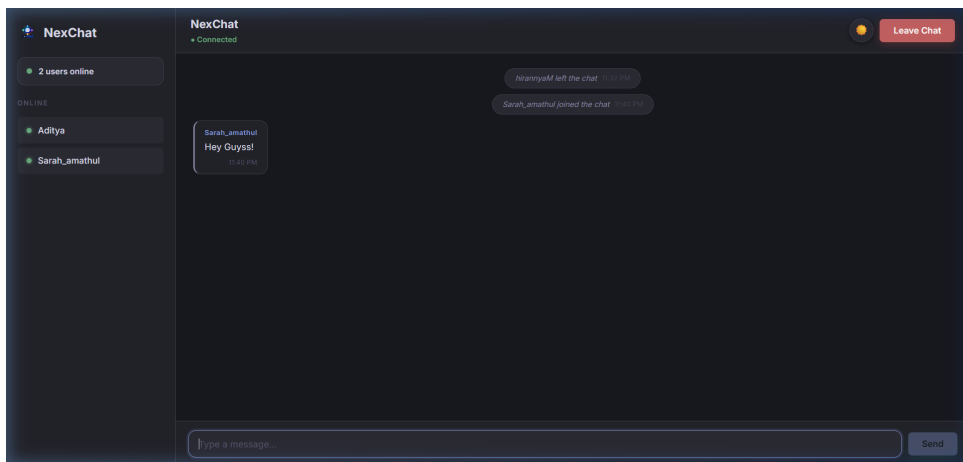


Figure 4: Step 3: All four users connected and online in the chat room.

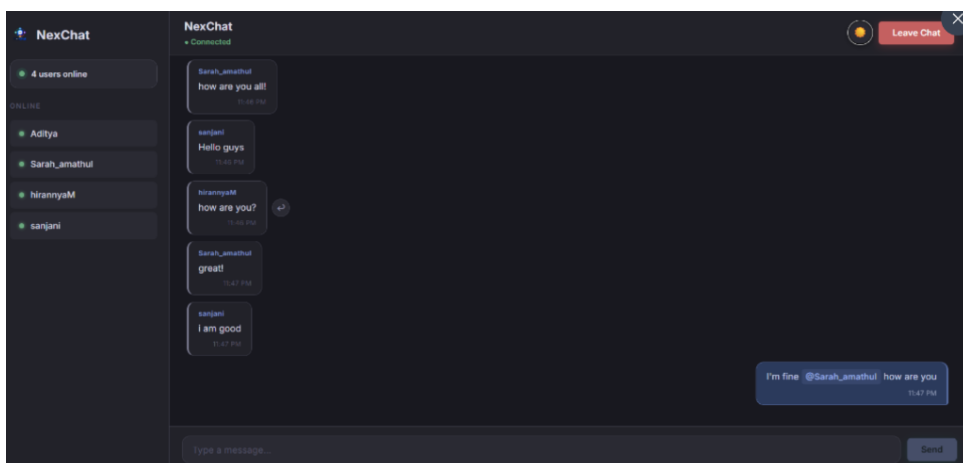


Figure 5: Step 4: Real messages exchanged between all four group members.

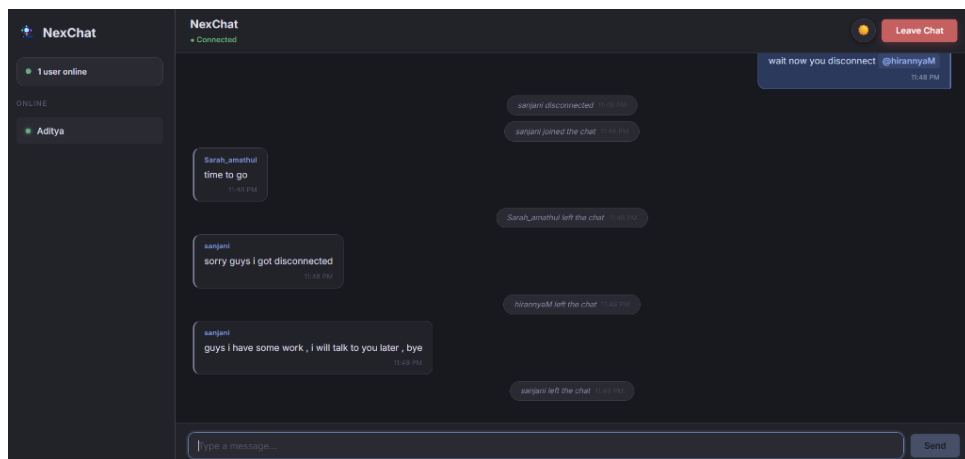


Figure 6: Step 5: System alerts as Sarah, Hirannya and Sanjani leave the chat one by one.

4 Tech Stack Used

The application utilizes a modern, decoupled technology stack designed for high concurrency and rapid development.

4.1 Frontend (Client-Side)

- **Framework:** React 18
- **Build Tool:** Vite (for fast Hot Module Replacement and optimized builds)
- **Styling:** Vanilla CSS3 (Custom Design System, CSS Variables)
- **Networking:** Native WebSockets (Custom `useWebSocket` React hook)
- **State/Persistence:** React Hooks + Browser `localStorage`
- **Deployment:** Vercel Global CDN

4.2 Backend (Server-Side)

- **Language:** Python 3
- **Framework:** FastAPI (Chosen for native `async/await` and WebSocket support)
- **Server:** Uvicorn (ASGI)
- **State Management:** In-Memory Python Dictionaries
- **Deployment:** Render Web Services

5 Architecture

NexChat follows a decoupled client-server architecture, enabling the frontend and backend to scale independently.

5.1 WebSocket Communication Pipeline

1. **Connection Handshake:** When the React app loads the `ChatRoom` component, the `useWebSocket` hook instantly establishes a connection to the FastAPI `/ws` endpoint, passing a persistent UUID generated via `crypto.randomUUID()`.
2. **Broadcasting Engine:** Every incoming message from a client is captured by the Python `async` loop, appended to a centralized `message_history` array, and immediately mapped over all active connections via `asyncio.gather`. This ensures absolute zero-latency delivery to all connected clients.

5.2 System State and Resilience

- **Session Tracking:** The React frontend runs a background `setInterval` heartbeat every 60 seconds to update the session's timestamp in `localStorage`.

- **Reconnection Logic:** On the backend, a `known_sessions` dictionary tracks the exact time a user disconnected. If a user rejoins within the 5-minute grace period, the server identifies them as a returning user and securely dumps the entire recent message history back to them.
- **Cross-Origin Resource Sharing (CORS):** Middleware is explicitly configured in FastAPI to safely accept connections from the external Vercel frontend domain, bridging the gap between the two cloud environments.

6 SSH Lab Deployment

In addition to the public cloud deployment, NexChat was successfully hosted and demonstrated across four SSH virtual machines in a university lab environment (server IP: 10.1.75.51), with each group member having a dedicated SSH port.

6.1 Host Machine Setup (Aditya — Port 2217)

One member hosted both the backend and the pre-built frontend:

1. **Clone and install backend dependencies:**

```
git clone https://github.com/Aj1359/Group-Chat-Application.git
cd Group-Chat-Application/backend
pip install -r requirements.txt
```

2. **Start the backend permanently in the background:**

```
nohup python3 -m uvicorn main:app \
  --host 0.0.0.0 --port 8000 > backend.log 2>&1 &
```

3. **Build the frontend locally** (on personal laptop), pointing it to `localhost:8000`:

```
echo "VITE_WS_URL=ws://localhost:8000/ws" > .env.production
npm run build
```

4. **Copy the built dist folder to the SSH machine:**

```
scp -P 2217 -r dist student@10.1.75.51:~/dist
```

5. **Serve the frontend permanently in the background:**

```
nohup python3 -m http.server 5173 \
  --directory ~/dist > frontend.log 2>&1 &
```

6.2 Connecting as a User (All Other Members)

Since the university firewall blocked direct IP access on non-standard ports, all other members connected via SSH port forwarding (tunneling). Each member ran the following

command on their **personal laptop**:

```
ssh -p <OWN_PORT> -L 5173:localhost:5173 \  
-L 8000:localhost:8000 student@10.1.75.51
```

This creates a secure tunnel that maps `localhost:5173` and `localhost:8000` on their personal laptop to the host VM's running services. After logging in, each member simply opened their browser at `http://localhost:5173` and entered the chat room, all connected to the same backend.

Member	SSH Port	Role
Aditya Jha	2217	Host (Backend + Frontend)
Hirannya Mhaisbadwe	2229	User (SSH Tunnel)
Sarah Amathul Azeez	2225	User (SSH Tunnel)
Sanjani Kumari	2221	User (SSH Tunnel)

Table 1: SSH port assignments for each group member

7 Conclusion

The development of NexChat successfully integrated a responsive React frontend with a high-performance Python backend. By applying core computer systems concepts to solve complexities like WebSocket state management, multi-tab synchronization, cross-origin deployment, and SSH port-forwarding in a restricted university network, the group delivered a reliable, secure, and visually polished real-time communication platform.